Davidmaraga.shop
Developer Integration Guide

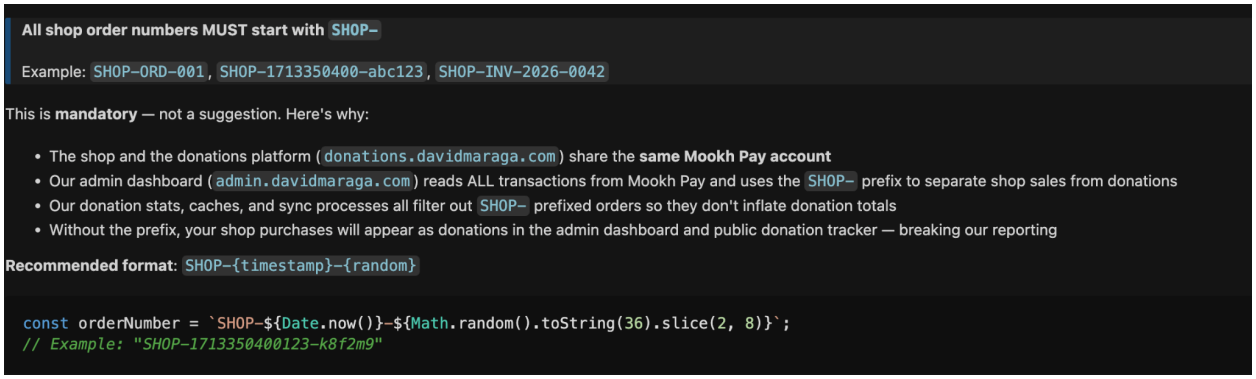
Project: davidmaraga.shop (E-commerce + Donation Gift Rewards)
Date: April 17, 2026

What You're Building

An e-commerce website at davidmaraga.shop that:

1. **Sells merchandise/items** with payments processed through Mookh Pay
2. **Rewards donors** - when someone donates on donations.davidmaraga.com, they can qualify for a gift from the shop based on how much they donated
3. **Verifies donations** - you'll query Mookh Pay to check a person's donation history and determine gift eligibility

CRITICAL: Order Number Prefix Requirement



Mookh Pay - The Payment Gateway

All payments (shop purchases AND donations) go through **Mookh Pay**. You will use their API to:

1. Accept payments on the shop (M-Pesa, cards, PayPal, etc.)
2. Query donation transactions to verify gift eligibility

Official Docs

1. **API Documentation**: https://mookhpay.docs.apiary.io
2. **Mookh Pay Website**: https://home.mookhpay.com/

Credentials

Will be shared separately via a secure channel. Format:

```
MOOKHPAY_USERNAME=<provided separately>
MOOKHPAY_PASSWORD=<provided separately>
```

Supported Payment Methods

Method	Currency	Type
M-Pesa	KES	Mobile Money (Kenya)
Bonga Points	KES	Mobile (Kenya)
Cards	KES, USD, RWF, UGX	Visa/Mastercard
PayPal	USD	Online
MTN Uganda	UGX	Mobile Money
Airtel Uganda	UGX	Mobile Money

API Endpoints

1. Authentication - Get Token

All API calls require a bearer token. Get one first, then cache it for ~15 minutes.

```
POST https://mookhpay.com/api/login

Content-Type: application/json

Body:
{
    "email": "<MOOKHPAY_USERNAME>",
    "password": "<MOOKHPAY_PASSWORD>"
}

Response:
{
    "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Use in all subsequent requests:

```
Authorization: Bearer <token>
```

Token tips:

1. Cache the token for ~15 minutes to avoid unnecessary auth calls
2. On 401 response, clear cache and re-authenticate

2. Initiate a Payment (Shop Purchase)

When a customer buys something from the shop:

```
POST https://mookhpay.com/api/checkout/init/payment

Authorization: Bearer <token>
Content-Type: application/json

Body:
{
    "amount": 2500,
    "currency": "KES",
    "pay_mode": "mpesa",
    "name": "Jane Doe",
    "phone": "254700000000",
    "email": "jane@example.com",
    "order_number": "SHOP-1713350400-k8f2m9",
    "account_reference": "MARAGA-SHOP",
    "callback_url": "https://davidmaraga.shop/api/payments/callback",
    "ipn_url": "https://davidmaraga.shop/api/payments/webhook"
}
```

Field notes:

1. pay_mode - one of: mpesa, card, paypal, bonga, MTNUG, AIRTELUG
2. order_number - **MUST start with SHOP-** (see prefix requirement above)
3. account_reference - shows on the customer's M-Pesa statement
4. callback_url - Mookh Pay POSTs payment result here when complete. Point this to YOUR shop domain
5. ipn_url - Instant Payment Notification, secondary webhook. Point this to YOUR shop domain

What happens next:

1. For M-Pesa: customer gets an STK push on their phone to enter PIN
2. For cards/PayPal: response includes a checkout_url - redirect the customer there
3. After payment, Mookh Pay calls your callback_url with the result

3. Query a Specific Order

Check status of a specific shop order:

```
GET https://mookhpay.com/api/checkout/init/query/payment/ref/{ref_id}

Authorization: Bearer <token>
```

Example: GET .../order/SHOP-1713350400-k8f2m9

4. Query by Payment Reference

GET https://mookhpay.com/api/checkout/init/query/payment/ref/{ref_id}

Authorization: Bearer <token>

5. Query Transactions (Search & Filter)

This is the endpoint you'll use most - both for shop order lookups and for checking donation history for the gift algorithm.

```
POST https://mookhpay.com/api/checkout/init/query/transactions

Authorization: Bearer <token>
Content-Type: application/json

Body (all fields optional — use any combination):
{
    "order_number": "SHOP-1713350400-k8f2m9",
    "currency": "KES",
    "payment_status": "paid",
    "payment_type": "...",
    "pay_mode": "mpesa",
    "name": "Jane",
    "phone": "254700000000",
    "amount": 2500,
    "payment_reference": "REF-ABC123"
}
```

Response (paginated):

```
{
    "data": [
        {
            "id": 123,
            "order_number": "SHOP-1713350400-k8f2m9",
            "amount": "2500.00",
            "currency": "KES",
            "payment_status": "paid",
            "pay_mode": "mpesa",
            "name": "Jane Doe",
            "phone": "254700000000",
            "email": "jane@example.com",
            "payment_reference": "REF-ABC123",
            "created_at": "2026-04-17T10:00:00.000000Z",
            "updated_at": "2026-04-17T10:01:00.000000Z"
        }
    ],
    "meta": {
        "current_page": 1,
        "last_page": 5,
        "per_page": 15,
        "total": 75
    },
    "links": {
        "next": "https://mookhpay.com/api/...?page=2",
        "prev": null
    }
}
```

Pagination: Response uses Laravel-style pagination. To get all pages, follow links.next until it's null, or loop from page 1 to meta.last_page.

Payment Flow (Step by Step)

```
Customer adds item to cart
        |
        v
Customer clicks "Pay" -> your frontend collects name, phone, email
        |
        v
Your backend: POST /api/login -> get token (or use cached token)
        |
        v
Your backend: POST /api/checkout/init/payment -> initiate payment
        |                                        (order_number MUST start with "SHOP-")
        |
        |-- M-Pesa: STK push sent to customer's phone
        |   |-- Customer enters PIN on phone
        |
        |-- Card/PayPal: redirect customer to checkout_url
        |   |-- Customer completes payment on Mookh Pay page
        |
        v
Mookh Pay calls YOUR callback_url with payment result
        |
        v
Your backend: update order status to "paid" / "failed"
        |
        v
Customer returns to your site -> show success/failure page
```

For M-Pesa specifically, poll for status while waiting:

1. After initiating, poll GET /api/checkout/init/query/payment/order/{order_number} every 3-5 seconds
2. Stop polling when payment_status changes from pending to paid or failed
3. Timeout after ~5 minutes

Donation-Gift Algorithm

This is the core integration between the shop and the donations platform.

How it works

Donations happen on donations.davidmaraga.com - all processed through the **same Mookh Pay account**. This means you can query Mookh Pay to see all donations made by a person.

How to check if someone qualifies for a gift

```
1. Customer provides their phone number or email on the shop

2. Query Mookh Pay for their donation history:
   POST https://mookhpay.com/api/checkout/init/query/transactions
   Body: {
       "phone": "254700000000",
       "payment_status": "paid"
   }

3. Filter results — EXCLUDE orders starting with "SHOP-"
   (those are shop purchases, not donations)

4. Sum up their total donations

5. Apply your gift threshold rules, e.g.:
   - KES 1,000+  -> small gift
   - KES 5,000+  -> medium gift
   - KES 10,000+ -> premium gift

6. Track gift redemptions in YOUR database to prevent double-claiming
```

Example implementation

```typescript
async function checkGiftEligibility(phone: string, token: string) {
  let totalDonated = 0;
  let page = 1;
  let lastPage = 1;

  while (page <= lastPage) {
    const res = await fetch(
      `https://mookhpay.com/api/checkout/init/query/transactions?page=${page}`,
      {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${token}`,
        },
        body: JSON.stringify({
          phone,
          payment_status: 'paid',
        }),
      }
    );

    const { data, meta } = await res.json();
    lastPage = meta.last_page;

    for (const txn of data) {
      // IMPORTANT: Skip shop purchases — only count donations
      if (txn.order_number?.startsWith('SHOP-')) continue;
      totalDonated += parseFloat(txn.amount);
    }

    page++;
  }

  // Apply threshold rules (define these based on business requirements)
  if (totalDonated >= 10000) return { eligible: true, tier: 'premium', totalDonated };
  if (totalDonated >= 5000)  return { eligible: true, tier: 'medium', totalDonated };
  if (totalDonated >= 1000)  return { eligible: true, tier: 'basic', totalDonated };

  return { eligible: false, tier: null, totalDonated };
}
```

How our system uses the SHOP- prefix

Our existing platform relies on the SHOP- prefix to keep things clean:

System	What it does with SHOP- orders
admin.davidmaraga.com	Shows them in a separate "Shop Sales" section - not counted as donations
donations.davidmaraga.com	Excludes them from public donation totals and donor stats
Donation cache	Filters them out so donation amounts stay accurate
Transaction sync	Skips them when syncing donations to our database
Webhooks	If a SHOP- webhook hits our donation endpoint, it's ignored

If you don't use the SHOP- prefix, your shop sales will appear as donations and inflate our numbers.

Distinguishing shop orders from donations

All transactions (shop + donations) go through the same Mookh Pay account:

Source	Order Number Pattern	How to identify
Shop purchases	SHOP-* (your orders)	Starts with SHOP-
M-Pesa donations	MP* or UUID	Does NOT start with SHOP-
Other donations	Various prefixes	Does NOT start with SHOP-

When calculating gift eligibility, **exclude** orders with SHOP- prefix - you only want to count donations.

Environment Variables for Your Project

```
# Mookh Pay (credentials shared separately)
MOOKHPAY_USERNAME=
MOOKHPAY_PASSWORD=

# Mookh Pay Endpoints
MOOKHPAY_AUTH_URL=https://mookhpay.com/api/login
MOOKHPAY_TRANS_URL=https://mookhpay.com/api/checkout/init/payment
MOOKHPAY_QUERY_ORDER_URL=https://mookhpay.com/api/checkout/init/query/payment/order
MOOKHPAY_QUERY_REF_URL=https://mookhpay.com/api/checkout/init/query/payment/ref
MOOKHPAY_QUERY_TRANSACTIONS_URL=https://mookhpay.com/api/checkout/init/query/transactions

# Your callback URLs (Mookh Pay sends payment results here)
# Point these to YOUR shop domain — NOT to donations.davidmaraga.com
MOOKHPAY_CALLBACK_URL=https://davidmaraga.shop/api/payments/callback
MOOKHPAY_IPN_URL=https://davidmaraga.shop/api/payments/webhook

# Your database
DATABASE_URL=postgresql://...
```

Things to Watch Out For

1. **Order number prefix** - Every order_number MUST start with SHOP-. This is non-negotiable. Without it, our entire reporting system breaks.
2. **Token expiry** - Cache the auth token for ~15 minutes. Re-authenticate on 401 errors.
3. **Response format variations** - Mookh Pay may return transaction data under different keys: data, transactions, or results. Handle all three:

const transactions = response.data || response.transactions || response.results || [ ];

1. **Rate limiting** - Add ~200ms delay between paginated requests when fetching multiple pages.
2. **Duplicate payments** - Check by phone number + amount + time window (~5 minutes) before creating new orders. Prevent customers from being charged twice.
3. **Webhook reliability** - Don't rely solely on webhooks. Also poll the order status endpoint as a fallback. Webhooks can be delayed or missed.
4. **Amount format** - Mookh Pay returns amounts as strings ("2500.00"). Always parseFloat() before doing math.
5. **Phone number format** - Use international format without +: 254700000000 (not 0700000000 or +254700000000).
6. **Order number uniqueness** - Every order_number must be unique across ALL transactions in the Mookh Pay account (including donations). Use: SHOP-{timestamp}-{random}.
7. **Callback URLs** - Point callback_url and ipn_url to YOUR shop domain. Do NOT point them to donations.davidmaraga.com - that would create false donation records.

Quick Reference Card

Action	Method	URL
Get auth token	POST	https://mookhpay.com/api/login
Start a payment	POST	https://mookhpay.com/api/checkout/init/payment
Check order status	GET	https://mookhpay.com/api/checkout/init/query/payment/order/{order_number}
Check by reference	GET	https://mookhpay.com/api/checkout/init/query/payment/ref/{ref_id}
Search transactions	POST	https://mookhpay.com/api/checkout/init/query/transactions

Search filters for transactions: order_number, currency, payment_status, payment_type, pay_mode, name, phone, amount, payment_reference

Pay modes: mpesa, card, paypal, bonga, MTNUG, AIRTELUG

Checklist Before Going Live

- ☐ All order numbers use SHOP- prefix
- ☐ Callback URLs point to your shop domain (not donations.davidmaraga.com)
- ☐ Token caching implemented (~15 min)
- ☐ Gift eligibility logic filters out SHOP- orders when summing donations
- ☐ Gift redemptions tracked in your database (no double-claiming)
- ☐ Duplicate payment prevention in place
- ☐ Webhook handler + polling fallback for payment status
- ☐ Phone numbers in 254XXXXXXXXX format

For Mookh Pay API issues: *https://mookhpay.docs.apiary.io* | *https://home.mookhpay.com/*

For project questions: *Mr. Rogers Chayuga*